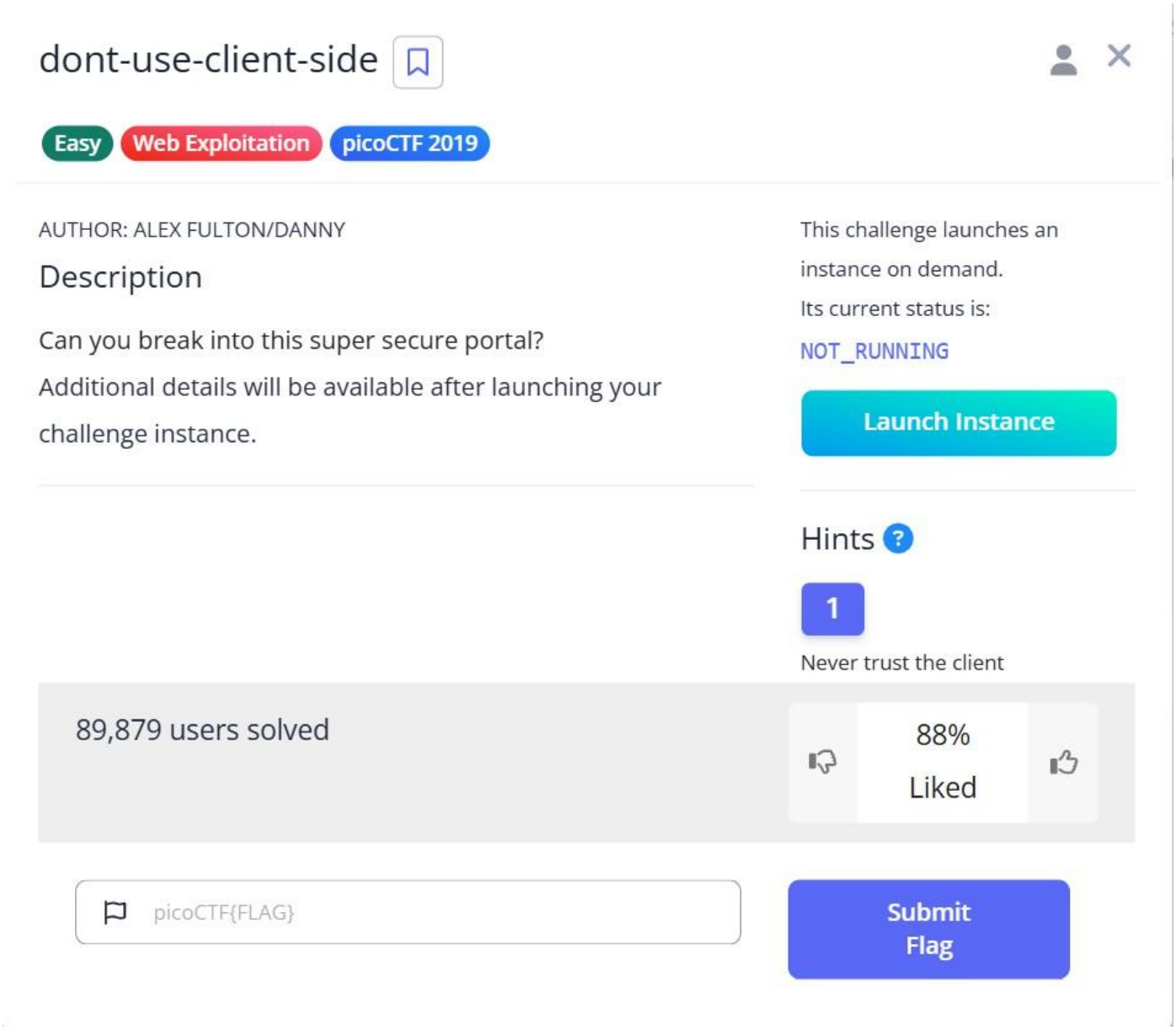
Dont-use-client-side



Author: The Analyst: Hyposelenia

Challenge: dont-use-client-side

Category: Web Exploitation

Date: 02/16/26

I. Objective

The objective of this challenge is to understand why sensitive logic and secrets should never be placed on the client side of a web application, and to demonstrate how inspecting client-side code can reveal hidden information such as flags.

II. Background

Web applications are divided into two main parts:

- **Client-side** – Code that runs in the user's browser (HTML, CSS, JavaScript)
- **Server-side** – Code that runs on the server and cannot be directly seen by users

A common security mistake is trusting client-side code to protect secrets. Since users can inspect, read, and modify client-side code, any sensitive data or logic placed there can be easily bypassed.

This challenge demonstrates why **client-side validation is not secure** and highlights a fundamental rule in web security:

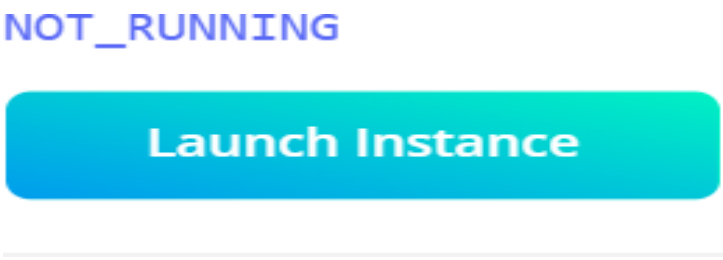
Never trust the client.

III. Tool Used

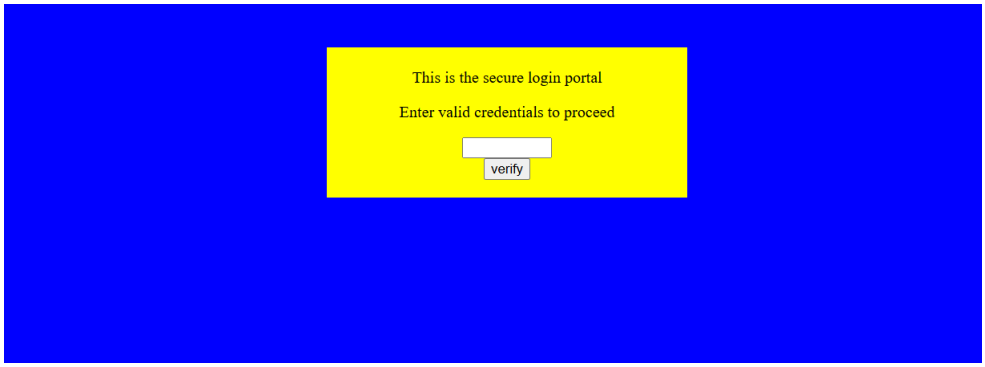
- **Web Browser (Google Chrome) –** Used to access the challenge website
- **Browser Developer Tools (Inspect) –** Used to view and analyze client-side code

IV. Methodology

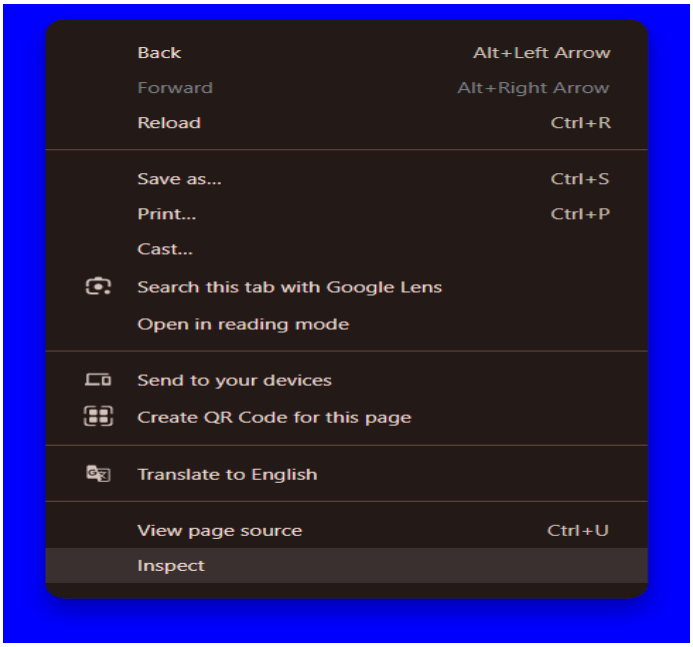
1. Launched the challenge instance.



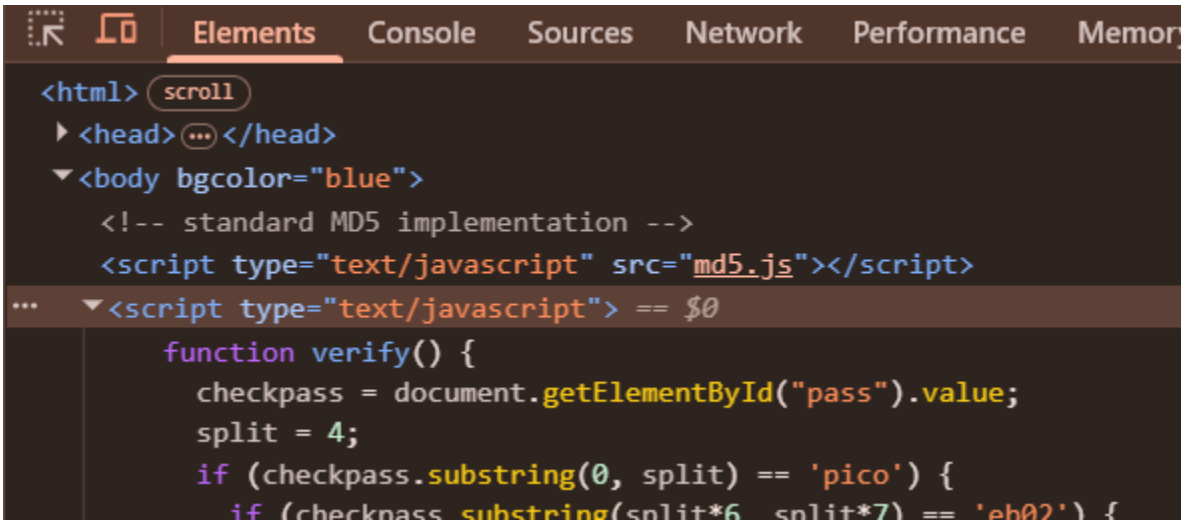
2. Copied the provided URL and opened it in a web browser.
3. Observed that the webpage appeared normal and required user interaction.



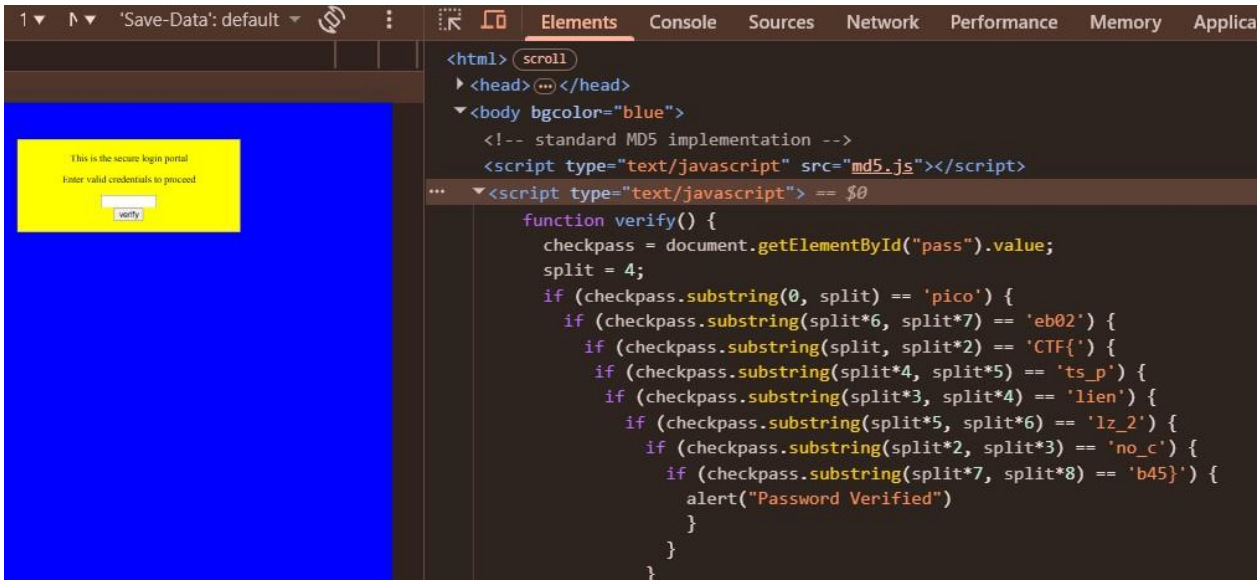
4. Right-clicked the page and selected **Inspect**, or pressed **F12**, to open Developer Tools.



5. Navigated to the **Elements** tab to examine the HTML structure.



6. Noticed a `<script>` tag containing JavaScript code within the page.

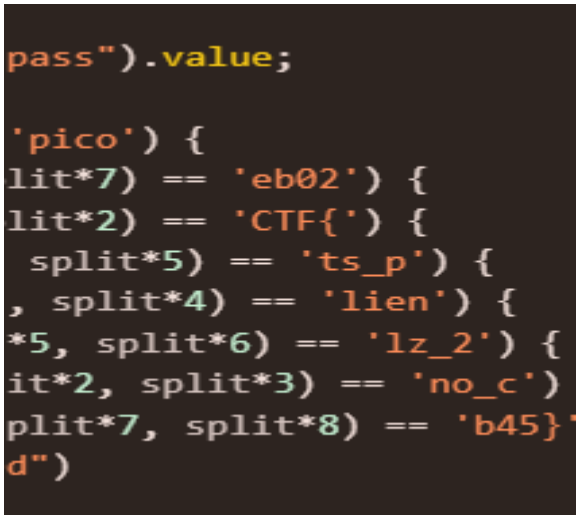


7. Located the `verify()` function inside the script.

```
function verify() {
  checkpass = document.getElementById("pass").value;
  split = 4;
  if (checkpass.substring(0, split) == 'pico') {
    if (checkpass.substring(split*6, split*7) == 'eb02') {
      if (checkpass.substring(split, split*2) == 'CTF{') {
       if (checkpass.substring(split*4, split*5) == 'ts_p') {
        if (checkpass.substring(split*3, split*4) == 'lien') {
          if (checkpass.substring(split*5, split*6) == 'lz_2') {
            if (checkpass.substring(split*2, split*3) == 'no_c') {
              if (checkpass.substring(split*7, split*8) == 'b45}') {
                alert("Password Verified")
                }
              }
            }
```

8. Observed that the function contained multiple `if` statements checking specific substrings.

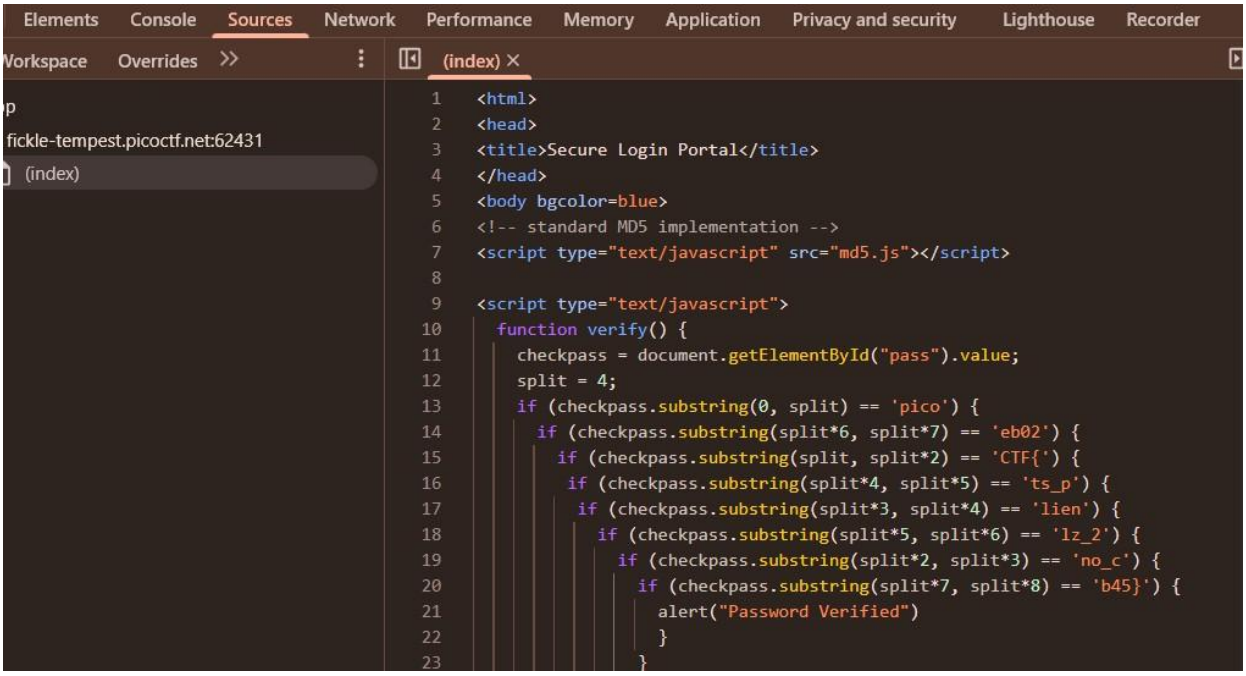
9. Noticed that each condition revealed a portion of the flag in sequence.



```
pass").value;

'pico') {
lit*7) == 'eb02') {
lit*2) == 'CTF{') {
 split*5) == 'ts_p') {
, split*4) == 'lien') {
*5, split*6) == 'lz_2') {
it*2, split*3) == 'no_c')
plit*7, split*8) == 'b45}'
d")
```

10. Reconstructed the flag by reading and combining all the string fragments shown in the conditions. (Go to sources to be able to highlight and copy the line of code)



```
Elements Console Sources Network Performance Memory Application Privacy and security Lighthouse Recorder
Workspace Overrides >>
(index) ×
p
fickle-tempest.picoctf.net:62431
(index)

<html>
<head>
<title>Secure Login Portal</title>
</head>
<body bgcolor=blue>
<!-- standard MD5 implementation -->
<script type="text/javascript" src="md5.js"></script>

<script type="text/javascript">
  function verify() {
    checkpass = document.getElementById("pass").value;
    split = 4;
    if (checkpass.substring(0, split) == 'pico') {
      if (checkpass.substring(split*6, split*7) == 'eb02') {
        if (checkpass.substring(split, split*2) == 'CTF{') {
          if (checkpass.substring(split*4, split*5) == 'ts_p') {
            if (checkpass.substring(split*3, split*4) == 'lien') {
              if (checkpass.substring(split*5, split*6) == 'lz_2') {
                if (checkpass.substring(split*2, split*3) == 'no_c') {
                  if (checkpass.substring(split*7, split*8) == 'b45}') {
                    alert("Password Verified")
                    }
                  }
```

11. Submitted the completed flag.

V. Result

picoCTF{no_clients_plz_2eb02b45}

VI. Explanation

Hmm… how should I put this… Imagine a **teacher writes the answers to a test on the blackboard** before students take it.

The teacher says, "*Don't look!*"

But the answers are still **right there**.

That's what client-side code is like. If the secret is sent to your browser, **you can always see it**.

Now let's analyze the code…

The website used JavaScript to "check" whether the correct password (or should we say flag) was entered.

However, instead of checking on the server, the entire flag was **broken into pieces inside the JavaScript code**.

Example idea (simplified):

if (input.substring(0, 4) == "pico") { ... }
if (input.substring(4, 8) == "CTF{") { ... }

Each condition exposed another part of the flag.

Because JavaScript runs **in the browser**, anyone can:

- Inspect it
- Read it
- Reverse it

- Bypass it

No hacking was needed — only **reading the code**.

Recalling what we've started, a client-side…

To explain simply:

- **Client** = Your browser (Chrome, Firefox, etc.)
- **Server** = The computer that owns the website

The client:

- Can see HTML
- Can see JavaScript
- Can change JavaScript
- Cannot be trusted

So now, we don't use client side for security?

Client-side code:

- Is visible to everyone
- Can be modified
- Cannot keep secrets

If passwords, flags, or validation logic are placed on the client side:

- Anyone can bypass checks
- Anyone can extract secrets
- The system becomes insecure

This challenge proves that **client-side checks are only for convenience, not security**.

Lastly, the purpose of this challenge teaches:

1. How to inspect client-side code
2. Why JavaScript cannot protect secrets
3. The danger of trusting user input

4. A core web security principle used in real-world hacking and defense

This vulnerability is known as:

Insecure Client-Side Validation

VII. Conclusion

The *dont-use-client-side* challenge demonstrates a fundamental web security mistake: trusting the client to protect sensitive logic. By inspecting the JavaScript code, the flag was easily reconstructed without any bypassing or brute-force attacks. This reinforces the importance of performing all security-critical checks on the server side. Understanding this concept is essential for both secure web development and effective web exploitation.

— **The Analyst: Hyposelenia**